



Bachelor Informatik/Wirtschaftsinformatik

Integrationsprojekt Kryptographie Teil 2

Spezifikationsdokument

Hannover, 8. Juni 2026

Sommerquartal 2026

Nina Neumann, Johanna Amini, Amirreza Ahmadi Darani, Xuan-Loc Tran, Franz Müller, Tammo Lütten, Jannes Breuer, Bjarne Möller, Darko Marjanovic, Oliver Knöbl, Rafael Ulmer

Anmerkung

Die vorliegende Arbeit wurde im Rahmen eines wissenschaftlichen Projekts an der Fachhochschule für die Wirtschaft (FHDW) Hannover erstellt. Alle in dieser Arbeit verwendeten Informationen, Daten und Ergebnisse sind ausschließlich für akademische Zwecke bestimmt. Die Autoren übernehmen keine Verantwortung für die Richtigkeit, Vollständigkeit oder Aktualität der in dieser Arbeit enthaltenen Informationen. Jegliche Nutzung der Inhalte dieser Arbeit erfolgt auf eigenes Risiko.

Erklärung der Selbstständigkeit

Hiermit versichern wir, dass wir das vorliegende Spezifikationsdokument selbstständig verfasst und keine anderen als die angegebenen Quellen verwendet haben. Alle Textstellen und andere Inhalte wie Bilder, Quellcode oder Daten, die wörtlich oder sinngemäß aus anderen Quellen entnommen sind, haben wir eindeutig mit korrekten Quellenangaben versehen. Über Zitierrichtlinien sind wir schriftlich informiert worden. Diese Arbeit wurde bisher in gleicher oder ähnlicher Form, auch nicht in Teilen, keiner anderen Prüfungsbehörde vorgelegt.

Falls wir während der Erstellung dieser Arbeit generative KI oder andere KI-gestützte Technologien verwendet haben, die über grundlegende Werkzeuge für Übersetzungen und zur Überprüfung von Grammatik oder Rechtschreibung hinausgehen, erklären wir, nach der Nutzung dieser Tools den Inhalt unseres Spezifikationsdokuments überprüft und bearbeitet zu haben und übernehmen die volle Verantwortung für die diesbezüglichen Ausführungen. Eine Verwendung solcher Werkzeuge haben wir in unserer Arbeit dokumentiert (bspw. im Abschnitt „Struktur der Arbeit, Methodik“ oder durch Erläuterungen zur Nutzung im Anhang).

Inhaltsverzeichnis

1 Einleitung

[?]

2 Querschnittssectionen

2.1 Architektur-Übersich

2.2 Gateway und Routing

2.3 Key Lifecycle

2.4 Serialisierung

2.5 Build, Deployment, Tests

3 Client

4 Use-Cases

4.1 Encrypted-Key-Value-Store

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.2 Encrypted-Age-Verification

Funktionsbeschreibung

Der Use Case Encrypted Age Verification löst das Problem, das Alter einer Person serverseitig zu prüfen, ohne dass der Server den tatsächlichen Alterswert jemals im Klartext zu Gesicht bekommt. Das Ergebnis der Prüfung (volljährig: ja/nein) wird ebenfalls verschlüsselt zurückgegeben - der Server kennt weder Eingabe noch Ausgabe.

Das Alter wird bereits auf dem Client mit dem ClientKey verschlüsselt und ausschließlich in verschlüsselter Form an das Backend übertragen. Das Backend führt die Altersüberprüfung mittels Fully Homomorphic Encryption (TFHE) durch, ohne den zugrunde liegenden Alterswert entschlüsseln zu können. Die Entschlüsselung des Ergebnisses ist ausschließlich durch den Besitzer des ClientKeys möglich.

Akteure

- Client: Generiert das TFHE-Schlüsselpaar, verschlüsselt das Alter mit dem ClientKey, sendet `encrypted_age` und `server_key` an den Server, empfängt das verschlüsselte Ergebnis und entschlüsselt es lokal. Der Client besitzt den ClientKey und ist der einzige Akteur, der das Ergebnis entschlüsseln kann.
- Backend (Server): Empfängt `encrypted_age` und `CompressedServerKey`, setzt den ServerKey, führt die homomorphe Altersüberprüfung (`age_check`) durch und gibt das verschlüsselte Ergebnis (`FheBool`) zurück. Der Server sieht zu keinem Zeitpunkt das Alter oder das Ergebnis im Klartext.

Lebenszyklus einer Session

Da dieser Use Case vollständig zustandslos ist, gibt es keine persistente Session. Der gesamte Ablauf findet innerhalb eines einzelnen HTTP-Requests statt:

1. Vorbereitung (Client): Client generiert `ClientKey` und `CompressedServerKey` lokal. Das Alter wird als `FheInt8` mit dem ClientKey verschlüsselt. Beide Werte werden bincode-serialisiert und base64-kodiert.
2. Request: Client sendet POST / mit `encrypted_age` und `server_key` als JSON-Body.
3. Verarbeitung (Server): Server dekodiert und deserialisiert beide Felder, setzt den ServerKey im globalen Thread-Kontext und führt `age_check` aus: `enc_age.gt(17) & enc_age.ge(0)`.

4. Response: Server serialisiert das FheBool-Ergebnis, base64-kodiert es und gibt es als `is_adult` zurück.
5. Auswertung (Client): Client dekodiert und deserialisiert `is_adult`, entschlüsselt das FheBool mit dem ClientKey und erhält das boolesche Ergebnis.

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.3 Encrypted-Voting-&-Polling

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.4 Sealed-Bid-Auction

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.5 Encrypted-Statistics-Service

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.6 Encrypted-Genomics

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.7 Encrypted-Image-Processing

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.8 Encrypted-Leaderboard

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.9 Encrypted-Program-Execution

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

A Anhang

Literatur

[Mus23] M. Mustermann. Example title. Online, 2023.